

SOC1 Fucker

Tartalom

Tartalom.....	2
Tipikus lépéssorrend a labor alatt	3
Alap-portok és helyek (gyakori alapértelmezett).....	3
1. Filebeat → Logstash → OpenSearch (adatbeküldés)	4
1.1 Filebeat konfiguráció (filebeat.yml)	4
1.2 Filebeat tesztelés és indítás	4
1.3 Logstash pipeline (input → filter → output)	5
1.4 Ellenőrzés: megérkezett-e az adat?	5
2. Logstash grok filter (strukturálás / parsing).....	6
2.1 Alap grok filter.....	6
2.2 Gyakori beépített grok minták	6
2.3 Saját (egyedi) minta	7
2.4 Grok tesztelése	7
2.5 Együtt használt filterek (date, mutate, geoip)	7
3. Riasztások OpenSearch-ben (Teams webhook).....	8
3.1 Teams Incoming Webhook URL beszerzése	8
3.2 Csatorna létrehozása (Notifications).....	8
3.3 Teams üzenet formátuma (JSON body).....	8
3.4 Monitor + trigger + action	8
4. Dashboardok készítése (OpenSearch Dashboards).....	10
4.1 Index pattern (előfeltétel).....	10
4.2 Vizualizáció létrehozása	10
4.3 Dashboard összeállítása	10
5. Logelemzés (kérdések megválaszolása)	11
5.1 DQL keresőszintaxis	11
5.2 Gyakori kérdések → hogyan válaszold meg	11
5.3 Dev Tools aggregációs lekérdezések	11
6. Forgalomelemzés (pcap fájl).....	13
6.1 Wireshark display filterek.....	13
6.2 Statistics menü (összesítő válaszok)	13
6.3 tshark (parancssor) megfelelők	13
6.4 Gyakori forgalomelemzési kérdések.....	14
7. Gyors hibakeresés	15
Függelék: minta Apache-sor parse-olása	15

Vizsga-munkamenet és gyors ellenőrzőlista

A teljes lánc: a logfájl a Filebeat olvassa, továbbküldi a Logstash-nek (5044-es port), a Logstash a grok filterrel strukturálja, majd az OpenSearch-be írja. Az OpenSearch Dashboardsban keletkeznek a vizualizációk, dashboardok és riasztások.

Adatút: logfájl → Filebeat → Logstash (input → filter/grok → output) → OpenSearch → OpenSearch Dashboards.

Tipikus lépéssorrend a labor alatt

1. Filebeat konfiguráció: input (logfájl elérési út) + output Logstash-re. Teszt: `filebeat test config` / `test output`.
2. Logstash pipeline: beats input (5044), grok filter, opensearch output. Szintaxis-teszt: `logstash -t`.
3. Filebeat indítása, majd ellenőrzés a Dashboards → Discover-ben (jön-e adat, jó-e az időtartomány).
4. Index pattern létrehozása, ha még nincs (Stack Management → Index Patterns).
5. Grok finomítása, amíg minden mező jól parse-olódik (nincs `_grokparsefailure` tag).
6. Dashboardok + riasztás (Teams webhook) összeállítása.
7. Log- és forgalomelemzési kérdések megválaszolása Discover / aggregáció / Wireshark segítségével.

Alap-portok és helyek (gyakori alapértelmezett)

Szolgáltatás	Port / hely	Megjegyzés
OpenSearch	9200 (HTTPS)	REST API, indexálás
OpenSearch Dashboards	5601	Webes felület (Discover, Dashboard, Alerting)
Logstash beats input	5044	ide küld a Filebeat
Filebeat konfiguráció	<code>/etc/filebeat/filebeat.yml</code>	csomagból telepítve
Logstash pipeline	<code>/etc/logstash/conf.d/*.conf</code>	több fájl is lehet

1. Filebeat → Logstash → OpenSearch (adatbeküldés)

Cél: a megadott logokat a Filebeat olvassa fel és továbbítja a Logstash-en keresztül az OpenSearch-be.

1.1 Filebeat konfiguráció (filebeat.yml)

A bemenet a logfájl(ok) elérési útja, a kimenet a Logstash. Az output.elasticsearch részt kapcsold ki (#), ha Logstash-en keresztül küldesz!

```
filebeat.inputs:
- type: log          # régebbi, egyszerű; vagy: filestream
  enabled: true
  paths:
  - /var/log/minta/*.log
  - /home/user/logok/access.log

# Kimenet: Logstash (NEM közvetlenül OpenSearch)
output.logstash:
  hosts: ["localhost:5044"]

# output.elasticsearch: <-- ezt kommenteld ki, ha Logstash-t hasznalsz
# hosts: ["localhost:9200"]
```

filestream típus (új logoknál ajánlott) – azonosítót (id) követel:

```
filebeat.inputs:
- type: filestream
  id: minta-logok
  enabled: true
  paths:
  - /var/log/minta/*.log
```

1.2 Filebeat tesztelés és indítás

Parancs	Mit csinál
filebeat test config	A yml szintaxisának ellenőrzése
filebeat test output	Eléri-e a Logstash-t (kapcsolat-teszt)
filebeat -e	Foreground futtatás, hibák a képernyőre (debug)
systemctl start filebeat	Szolgáltatásként indítás
systemctl status filebeat	Állapot és legutóbbi log
journalctl -u filebeat -f	Élő napló követése

FONTOS – újraküldés: a Filebeat megjegyzi, meddig olvasott (registry). Ha ugyanazt a fájlt az elejétől újra be akarod küldeni, állítsd le, töröld a registryt, majd indítsd:

```
sudo systemctl stop filebeat
sudo rm -rf /var/lib/filebeat/registry    # csomagos telepítés
```

```
# vagy tar-bol futtatva: rm -rf data/registry
sudo systemctl start filebeat
```

1.3 Logstash pipeline (input → filter → output)

Egy .conf fájl három blokkból áll. A beats input fogadja a Filebeatet, az opensearch output írja ki az indexet. (A filter/grok részt lásd a 2. fejezetben.)

```
input {
  beats {
    port => 5044
  }
}

filter {
  # ide jön a grok stb. (2. fejezet)
}

output {
  opensearch {
    hosts => ["https://localhost:9200"]
    index => "minta-logok-%{+YYYY.MM.dd}"
    user => "admin"
    password => "admin"
    ssl => true
    ssl_certificate_verification => false
  }
  stdout { codec => rubydebug } # debug: a parse-olt esemény a konzolra
}
```

Logstash hasznos parancsok:

Parancs	Mit csinál
bin/logstash -t -f pipeline.conf	Csak szintaxis-ellenőrzés (config test)
bin/logstash -f pipeline.conf	Pipeline futtatása kézzel (teszthez jó)
systemctl restart logstash	Szolgáltatás újraindítása config-módosítás után
tail -f /var/log/logstash/logstash-plain.log	Logstash saját hibáinak figyelése

1.4 Ellenőrzés: megérkezett-e az adat?

- Dashboards → Discover: válaszd ki az index pattern-t, állítsd az időtartományt elég szélesre (pl. Last 1 year).
- Parancssorból az indexek listázása:

```
curl -k -u admin:admin https://localhost:9200/_cat/indices?v
curl -k -u admin:admin "https://localhost:9200/minta-logok-*/_count"
```

2. Logstash grok filter (strukturálás / parsing)

A grok minta a nyers message mezőből névvel ellátott mezőket készít. Szintaxis:

%{MINTA:mezőnev}. Egy mintaelem mindig egy beépített (vagy saját) regexet takar.

2.1 Alap grok filter

```
filter {
  grok {
    match => { "message" => "%{IP:kliens_ip} %{WORD:metodus}
%{URIPATHPARAM:utvonal} %{NUMBER:statusz}" }
  }
}
```

Több minta egy mezőre (első találat számít) – eltérő formátumú sorokhoz:

```
grok {
  match => { "message" => [
    "%{COMBINEDAPACHELOG}",
    "%{SYSLOGTIMESTAMP:idobelyeg} %{WORD:folyamat}:"
  ] }
}
```

2.2 Gyakori beépített grok minták

Minta	Mit fog meg
%{IP} / %{IPV4} / %{IPV6}	IP cím
%{IPORHOST}	IP vagy hosztnév
%{NUMBER} / %{INT}	Szám (tizedes is) / egész
%{WORD}	Betű/szám karaktersor (szóköz nélkül)
%{NOTSPACE}	Minden, ami nem szóköz
%{DATA}	Nem mohó tetszőleges (legrövidebb)
%{GREEDYDATA}	Mohó tetszőleges (sor végéig)
%{QUOTEDSTRING}	Időzőjelek közötti szöveg
%{TIMESTAMP_ISO8601}	2026-06-04T12:00:00 formátumú idő
%{HTTPDATE}	Apache időbélyeg: 04/Jun/2026:12:00:00 +0200
%{SYSLOGTIMESTAMP}	Jun 04 12:00:00 (syslog)
%{LOGLEVEL}	INFO, ERROR, WARN stb.
%{URIPATH} / %{URIPATHPARAM}	URL útvonal / paraméterekkel
%{USER} / %{USERNAME}	Felhasználónév
%{COMBINEDAPACHELOG}	Teljes Apache access sor egyben

2.3 Saját (egyedi) minta

Ha nincs készen a minta, írhatod sajátot soron belül regexszel, vagy névvel:

```
# soron belüli egyedi minta: (?<mezonev>regex)
grok { match => { "message" => "(?<azonosito>[A-Z]{3}-[0-9]{4})" } }

# vagy nevesített mintakönyvtárból:
grok {
  pattern_definitions => { "AZONOSITO" => "[A-Z]{3}-[0-9]{4}" }
  match => { "message" => "%{AZONOSITO:azonosito}" }
}
```

2.4 Grok tesztelése

- Dashboards → Dev Tools → Grok Debugger (ha elérhető): beilleszted a minta-sort és a mintát, látod a parse-olt mezőket.
- Iteratív módszer: tedd be a `stdout { codec => rubydebug }` sort az outputba, indítsd kézzel a Logstash-t, nézd a konzolt. Így látod a mezőket és a `_grokparsefailure` taget.
- Tipp: ha nem illeszkedik, gyakori ok a több szóköz, a hiányzó `[ ]` vagy a túl szűk minta (WORD helyett próbálj NOTSPACE / DATA elemet).

2.5 Együtt használt filterek (date, mutate, geoip)

A grok után szokás a kinyert időbélyeget `@timestamp`-re állítani, típust konvertálni és földrajzi adatot hozzáadni.

```
filter {
  grok { match => { "message" => "%{COMBINEDAPACHELOG}" } }

  date {
    match => ["timestamp", "dd/MMM/yyyy:HH:mm:ss Z"]
    target => "@timestamp"
  }

  mutate {
    convert => { "response" => "integer" "bytes" => "integer" }
    remove_field => ["message"] # nyers sor eldobása (opcionális)
  }

  geoip { source => "clientip" } # ország/varos a kliens IP-ből
}
```

mutate gyakori opciók: `convert` (típus), `rename` (átnevezés), `remove_field` (törlés), `add_field` (hozzáadás), `gsub` (csere regexszel), `lowercase` / `uppercase`.

Hibakezelés: ha a grok nem illeszkedik, az esemény `_grokparsefailure` taget kap. Szűrd ki Discoverben: `tags:_grokparsefailure`. Saját tag: `tag_on_failure => ["saját_hiba"]`.

3. Riasztások OpenSearch-ben (Teams webhook)

Két lépés: (1) csatorna (channel) létrehozása a Teams webhook URL-lel a Notifications modulban, (2) monitor + trigger + action a Alerting modulban, amely erre a csatornára küld.

3.1 Teams Incoming Webhook URL beszerzése

- Teams csatornában: ... (több opció) → Connectors / Workflows → Incoming Webhook → név + létrehozás → másold ki az URL-t (<https://...webhook.office.com/...>).
- Ezt az URL-t fogja meghívni az OpenSearch a riasztáskor.

3.2 Csatorna létrehozása (Notifications)

- Dashboards → Notifications → Channels → Create channel.
- Channel type: Custom webhook (univerzális), vagy Microsoft Teams, ha az adott verzióban külön létezik.
- Custom webhook esetén: Method = POST, illeszd be a teljes Teams URL-t, header: Content-Type = application/json.
- Send test message gombbal ellenőrizd, hogy beérkezik-e a Teams csatornába.

3.3 Teams üzenet formátuma (JSON body)

A Teams a JSON body text mezőjét jeleníti meg. A monitor action üzenetébe ezt a JSON-t írd (mustache változókkal):

```
{
  "text": "Riasztas: {{ctx.monitor.name}} | {{ctx.trigger.name}} | talalatok:
  {{ctx.results.0.hits.total.value}}"
}
```

Új Teams (Workflows) Adaptive Card formátumot várhat – ha a sima text nem jelenik meg, próbáld:

```
{
  "type": "message",
  "attachments": [{
    "contentType": "application/vnd.microsoft.card.adaptive",
    "content": {
      "type": "AdaptiveCard", "version": "1.4",
      "body": [{ "type": "TextBlock", "text": "Riasztas: {{ctx.monitor.name}}" }]
    }
  }]
}
```

3.4 Monitor + trigger + action

- Dashboards → Alerting → Monitors → Create monitor.
- Monitor típus: Per query (egyszerű számláláshoz) vagy visual editor. Állítsd be az indexet, az időmezőt (@timestamp) és az ütemezést (pl. percenként).
- Definiáld a lekérdezést/metrikát (pl. status:500 száma az utóbbi 5 percben).

Trigger feltétel (Painless / extraction query monitornál):


```
ctx.results[0].hits.total.value > 5
```

- Action: válaszd ki a 3.2-ben létrehozott csatornát, add meg a Subject-et és a Message-et (a 3.3 JSON body).
- Mentsd el. Teszteléshez generálj annyi találatot, hogy a feltétel teljesüljön, vagy használd a Send test message-et a csatornánál.

Hasznos ctx változók: `{{ctx.monitor.name}}`, `{{ctx.trigger.name}}`,
`{{ctx.results.0.hits.total.value}}`, `{{ctx.periodStart}}`, `{{ctx.periodEnd}}`.

4. Dashboardok készítése (OpenSearch Dashboards)

4.1 Index pattern (előfeltétel)

- Stack Management → Index Patterns / Dashboards Management → Create index pattern.
- Add meg a mintát (pl. minta-logok-*), válaszd időmezőnek a @timestamp-et.
- Ha új mező nem látszik: nyisd meg a patternt → Refresh field list.

4.2 Vizualizáció létrehozása

- Visualize → Create visualization → válassz típust, majd az index patternt.

Tipikus típusok és mire jók:

Vizualizáció	Mire jó	Tipikus beállítás
Vertical/Horizontal bar	Top N összehasonlítás	Y: Count, X: Terms (mező.keyword)
Pie / Donut	Arányok	Slice size: Count, Split: Terms
Data table	Pontos számok táblázatban	Metric: Count, Bucket: Terms
Metric	Egyetlen szám (pl. összes esemény)	Metric: Count / Unique count
Line / Area	Időbeli trend	Y: Count, X: Date histogram (@timestamp)
Region map / coords	Földrajzi eloszlás	geoip mező alapján

FONTOS: szöveges (text) mezőre az aggregáció a .keyword változatot használja (pl. clientip.keyword), különben nem csoportosítható.

4.3 Dashboard összeállítása

- Dashboard → Create → Add → vedd fel a mentés után a kész vizualizációkat.
- Átméretezés és átrendezés húzással; fent állíthatod a globális időtartományt és szűrőket.
- Save: adj nevet – a kiértékelő ez alapján találja meg a munkádat.

5. Logelemzés (kérdések megválaszolása)

A Discover keresősávja DQL (Dashboards Query Language) szintaxist használ. Összetett kérdésekhez aggregáció (Data table / Metric) vagy Dev Tools `_search` a leggyorsabb.

5.1 DQL keresőszintaxis

Példa	Jelentés
status:404	A status mező értéke 404
status:404 and method:POST	Mindkét feltétel
status:404 or status:500	Bármelyik
not status:200	Kizárás
response >= 500	Tartomány (számra)
url:*.php	Helyettesítő karakter
clientip:"10.0.0.5"	Pontos érték idézőjelben
user:*	Létezik-e a mező (bármilyen érték)
message:login	A message tartalmazza: login
tags:_grokparsefailure	Rosszul parse-olt sorok

5.2 Gyakori kérdések → hogyan válaszold meg

Kérdés	Módszer
Hány esemény van összesen?	Discover találatszám, vagy Metric (Count)
Melyik IP a leggyakoribb?	Data table / bar: Terms clientip.keyword, Count, rendezés
Hány sikertelen bejelentkezés?	DQL szűrés (pl. event:failed_login) + találatszám
Melyik a legtöbbet kért URL?	Terms url.keyword aggregáció, top 1
Hány egyedi felhasználó?	Metric → Unique count (cardinality) user.keyword
Mikor volt az első/utolsó esemény?	Discover rendezés @timestamp szerint (asc/desc)
Mely órában van csúcs?	Line/bar: Date histogram @timestamp
Milyen státuszkódok jöttek?	Pie / table: Terms status

5.3 Dev Tools aggregációs lekérdezések

Top 10 forrás IP:

```
GET minta-logok-*/_search
{
  "size": 0,
  "aggs": {
    "top_ip": { "terms": { "field": "clientip.keyword", "size": 10 } }
  }
}
```

```
}  
}
```

Egyedi felhasználók száma + szűrt darabszám:

```
GET minta-logok-*/_search  
{  
  "size": 0,  
  "query": { "match": { "status": 500 } },  
  "aggs": { "egyedi_user": { "cardinality": { "field": "user.keyword" } } }  
}
```

6. Forgalomelemzés (pcap fájl)

A pcap-et Wiresharkkal (grafikus) vagy tshark-kal (parancssor) elemzed. A megjelenítési szűrő (display filter) szűki a látható csomagokat; a Statistics menü adja az összesítő válaszokat.

6.1 Wireshark display filterek

Szűrő	Mit mutat
ip.addr == 10.0.0.5	Adott IP forrásként vagy célként
ip.src == x / ip.dst == x	Csak forrás / csak cél
tcp.port == 80 / udp.port == 53	Adott port
http / dns / tls / icmp / arp	Protokoll szűrés
http.request.method == "POST"	POST kérések
http.request.uri contains "login"	URI-ban keresés
http.host == "pelda.hu"	Adott weboldal
dns.qry.name contains "google"	DNS lekérdezés neve
tcp.flags.syn==1 && tcp.flags.ack==0	SYN csomagok (port scan jele)
frame contains "password"	Szöveg keresése bárhol a csomagban
tcp.stream eq 0	Egy adott TCP-folyam
ip.addr==a && ip.addr==b	Két fél közötti forgalom

6.2 Statistics menü (összesítő válaszok)

Menüpont	Mire válaszol
Capture File Properties	Összes csomag száma, időtartam
Protocol Hierarchy	Milyen protokollok, milyen arányban
Conversations	Mely IP/port párok beszélnek, byte/csomag szerint
Endpoints	Melyik IP küldte/kapta a legtöbb forgalmat
IO Graphs	Forgalom időbeli alakulása

Fájl/objektum kinyerése: File → Export Objects → HTTP – itt lementők a forgalomban átvitt fájlok. Jelszavak/adatok: jobb klikk egy csomagon → Follow → TCP/HTTP Stream.

6.3 tshark (parancssor) megfelelők

```
# HTTP kérések forrás IP + host + URI
tshark -r f.pcap -Y "http.request" -T fields -e ip.src -e http.host -e http.request.uri

# Protokoll-hierarchia (összesítés)
tshark -r f.pcap -q -z io,phs

# TCP beszélgetések (ki kivel, mennyit)
```

```
tshark -r f.pcap -q -z conv,tcp
```

```
# DNS lekerdezések nevei
```

```
tshark -r f.pcap -Y "dns.flags.response == 0" -T fields -e dns.qry.name
```

```
# Szoveg keresese (pl. jelszo)
```

```
tshark -r f.pcap -Y 'frame contains "password"'
```

6.4 Gyakori forgalomelemzési kérdések

Kérdés	Hogyan
Hány csomag van?	Statistics → Capture File Properties
Melyik IP a legaktívabb?	Statistics → Endpoints, rendezés byte szerint
Milyen protokollok vannak?	Statistics → Protocol Hierarchy
Milyen oldalakat látogattak?	http.host és dns.qry.name szűrők
Van-e jelszó/hitelesítő adat?	Follow HTTP Stream / frame contains
Töltöttek-e le fájlt?	File → Export Objects → HTTP
Volt-e port scan?	tcp.flags.syn==1 && ack==0, sok célport egy IP-ről
Hány csomag illik a szűrőre?	Szűrő alkalmazása → alul: Displayed szám
Milyen User-Agent?	http.user_agent mező

7. Gyors hibakeresés

Tünet	Valószínű ok / megoldás
Nincs adat az OpenSearch-ben	Fut-e a Filebeat? filebeat test output. Logstash figyel-e 5044-en? Jó-e az időtartomány Discoverben? Létezik-e az index pattern?
_grokparsefailure tag	A grok minta nem illeszkedik. Teszteld a debuggerben; figyelj a szóközökre; próbálj DATA/NOTSPACE elemet.
A Filebeat nem küld újra	EOF a registryben. Állítsd le, töröld a registryt, indítsd újra.
Riasztás nem tüzel	Trigger feltétel/küszöb, ütemezés, vagy hibás webhook URL. Send test message a csatornán.
Teams-be nem jön üzenet	Rossz JSON body. Sima {"text":"..."}; új Teamsnél Adaptive Card. Content-Type: application/json.
Mezőre nem megy aggregáció	Szöveges mező: használd a .keyword változatot. Új mező: Refresh field list.
Logstash nem indul	logstash -t a config-hibáért; nézd a logstash-plain.log-ot.

Függelék: minta Apache-sor parse-olása

Egy teljes példa, amely összeköti a fejezeteket: nyers Apache access sor → mezők.

Nyers sor:

```
10.0.0.5 - - [04/Jun/2026:12:00:00 +0200] "GET /index.php HTTP/1.1" 200 1024
```

Logstash filter:

```
filter {
  grok { match => { "message" => "%{COMBINEDAPACHELOG}" } }
  date { match => ["timestamp", "dd/MMM/yyyy:HH:mm:ss Z"] target => "@timestamp"
}
mutate { convert => { "response" => "integer" "bytes" => "integer" } }
}
```

Eredmény mezők: clientip=10.0.0.5, verb=GET, request=/index.php, response=200, bytes=1024, és a @timestamp a sor idejére állítva.